

# 実装の講評と総括

判断と証拠を振り返る

情報システム学科 3年・選択・2単位

# 今日の到達目標

- 実装の達成範囲を証拠で説明できる
- 他者の設計を問い、改善案を示せる
- AI利用を含めて学習と開発の変化を振り返れる

# 前回との接続

Lisp製評価器とREPLをPython製ホスト上で実行しました。

今日は新機能の追加より、開発したものの講評を行います。

実装の現在地と次に改善する点を確認めます。

# 講評の対象

コードだけでなく、仕様、受入例、実行ログ、説明を扱います。

動いた結果と、その結果になる理由を結び付けます。

ここでの観点は授業用の案であり、正式な点数配分ではありません。

# 講評の進め方

作者は達成した機能と、未対応の機能を説明します。

聞き手は小さな入力で判断を確かめます。

作者は指摘を受け、改善を1つ選びます。

ペアで作者と聞き手を交代します。共通講評の後も、各人のデモと質問の時間を残します。

# 架空の提出例

以後の例は、本教材が作成した架空の実装です。

実在の学生のコード・提出物・成績は使用しません。

例題 `examples/15-showcase/` を議論の材料にします。

# 例A 算術だけの実装

四則演算と入れ子の式が動きます。

変数と関数は未対応ですが、失敗条件をREADMEに記載しました。

成功例と失敗例から、現在の達成範囲を読み取ってください。

# 例Aへの問い

`(+ 1 (* 2 3))` の評価順序を説明できるか。

未知の演算子を渡すと、利用者は原因を理解できるか。

次に変数を加えるなら、どの関数へ環境を渡すか。

# 例B 環境の混同

関数の呼出しごとに、同じ変数表へ引数を書き込む架空の案です。

`f` の引数が、外側の変数を上書きしてしまいます。

正常な一例だけではこの誤りを見落とします。

# 例Bの反例

```
(define x 100)
(define f (lambda (x) (+ x 1)))
(f 4)
x
```

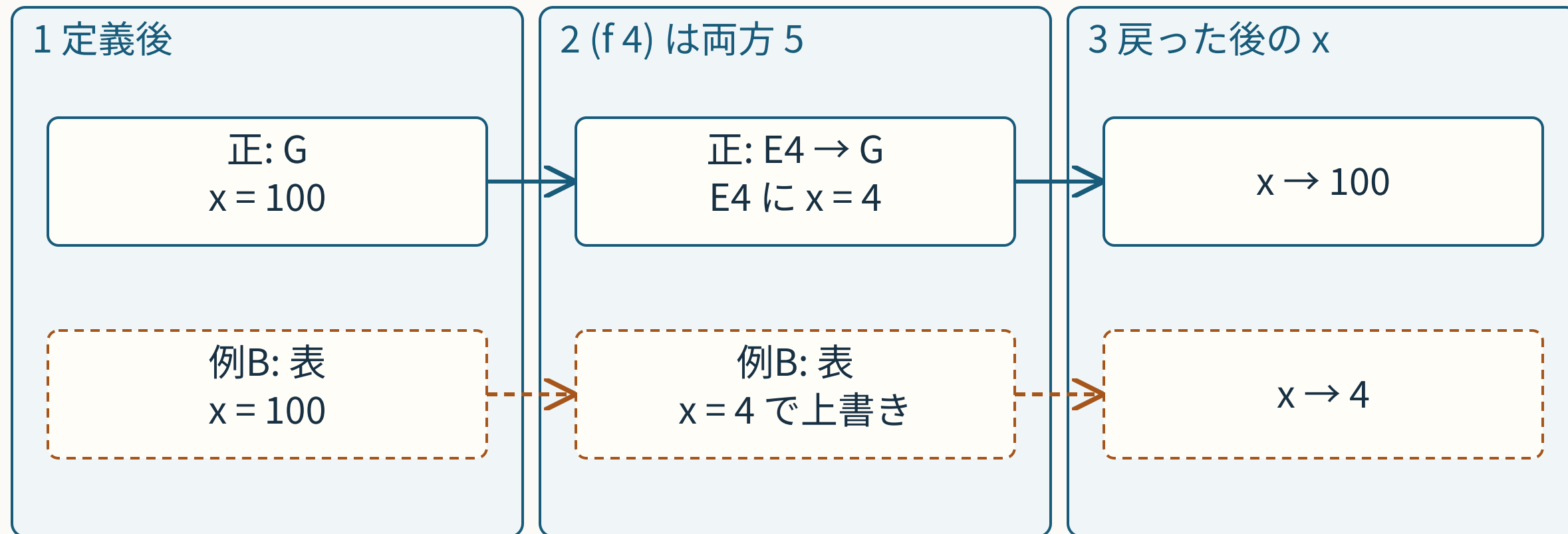
本教材の仕様では結果は 5 と 100 です。

作者は検索する環境と更新する環境を説明します。

例Bの単一表の案で、(f 4) の後に x を評価した値を予想してください。

## 例B: 単一表は外側を壊す

正しい実装は呼出しごとに子環境を作り、例Bは同じ表に  $x=4$  を書きます。



別入力: 例Bを見落とさないテストを1つ作り、作者役に結果を予想させてから実行します。

# 例C Lisp製評価器

対象の `if` と関数をLispで評価できる架空の案です。

読取と入出力はPythonホストへ委譲しています。

対応機能と委譲範囲を図と実行ログで示しました。

# 例Cへの問い

Lisp製評価器を呼ぶコードはどこか。

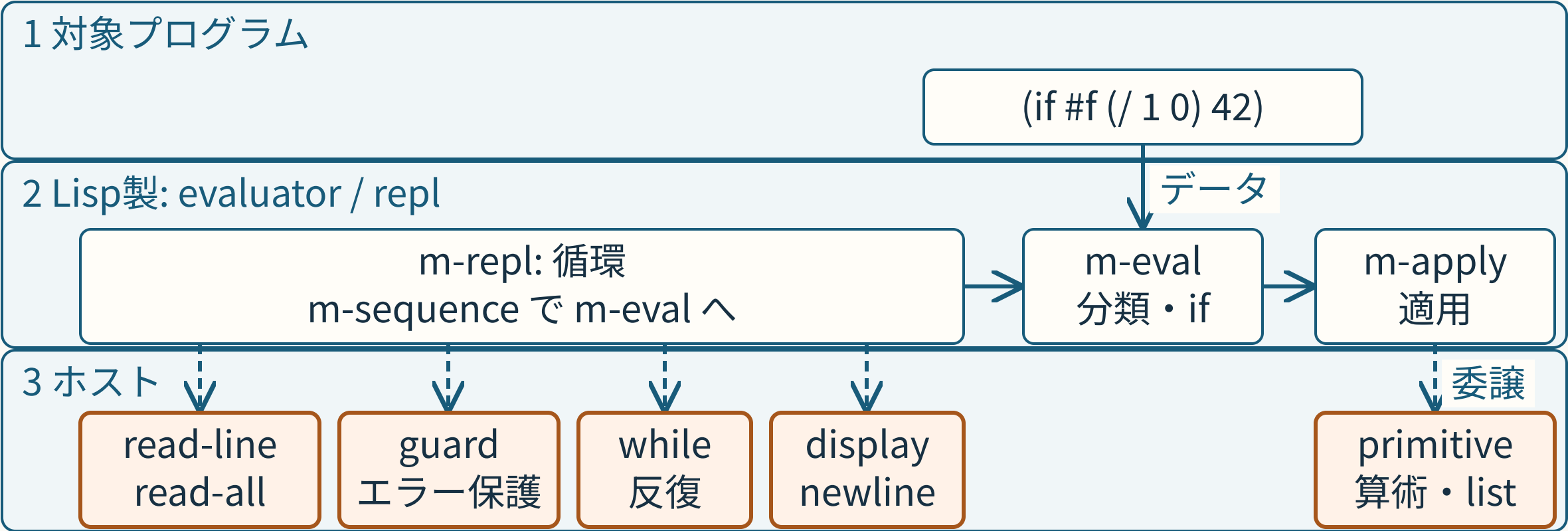
非選択枝を評価しない証拠はあるか。

ホストへの依存を残した理由と、その影響を説明できるか。

参照実装で『ホストへの依存』に当たる処理を1つ予想してください。

# 参照実装の層と委譲

式の意味は Lisp 側が決め、読取・算術・入出力・保護・反復はホストです。



確認: 作者は自分の実装で同じ層の図を描き、ホストへ委譲した処理を1つずつ指します。

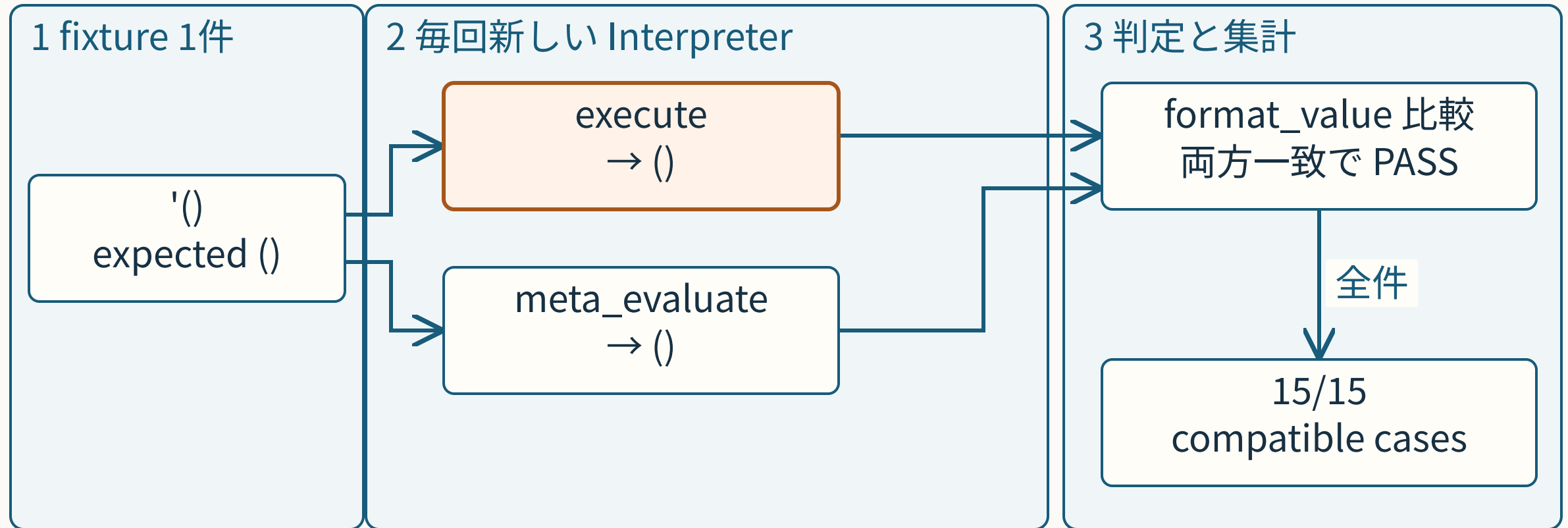
# 証拠の比較

| 証拠   | 分かること   |
|------|---------|
| 実行ログ | 観察した動作  |
| 受入例  | 守る契約    |
| 環境図  | 名前検索の理由 |
| コード  | 処理の実装箇所 |

同じ fixture をホストと Lisp製評価器で実行したとき、PASS になる条件を予想してください。

# 互換 fixture の確認経路

1件ごとに新しい Interpreter で host と meta を実行し、両方一致で PASS です。



別入力: fixture の `(car (list))` で、host と meta が出す診断語を予想し実行で照合します。

# 自分の実装の紹介

対応する入力を1つ選び、結果を予測してから実行してください。

その結果を作るコードと環境を説明してください。

未対応入力も1つ示し、診断か制限を述べてください。

# 聞き手の質問

作者の説明で曖昧な名前や順序を1つ選んでください。

小さな反例を提案し、予測を聞いてください。

作者が実行して確かめられる問いにしてください。

# 講評の記録

観察した事実と、改善の提案を分けて書いてください。

「動かない」だけで終わらせず、入力と実際の結果を残してください。

作者の意図を確認してから、設計判断を評価してください。

# 設計の振り返り

変更が少ない箇所と、変更が広がった箇所を比べてください。

読取、評価、環境、入出力の境界は役立ったか。

1つの変更を例に、自分の判断を説明してください。

# データ構造との接続

構文木は再帰的なデータ構造として扱いました。

環境探索は、階層をたどる探索になりました。

前半の木、探索、計算量がどこで役立ったか挙げてください。

# 検証の振り返り

最初に見落とした反例を1つ選んでください。

どの契約が曖昧だったかを書き直してください。

その反例を、今後も使う回帰検証へ残してください。

# AI利用の振り返り

AIへ任せた作業と、自分が判断した部分を分けてください。  
採用前に何を確認したか、具体的な入力で示してください。  
説明を自分で修正した例も振り返ってください。

# 改善案の選択

実装の改善を1つ選び、利用者に起きる変化を書いてください。

変更箇所と、成功を確かめる受入例を示してください。

未実装の案は、完成済みの機能と分けて記述してください。

# 確認問い

結果が正しい一例だけで、設計の正しさを主張できるか。

自分の実装を他者へ渡すとき、どの証拠を一緒に渡すか。

# 成果物の整理

コード、仕様、受入例、実行ログ、振り返りを対応付けてください。

教材の成果物案は `exercises/15.md` にあります。

正式な提出先・締切・採点運用は授業で確認してください。

# 全体の総括

ソフトウェアの設計では、必要な動作を契約にし、変更を扱う構造を作ります。

プログラムを書き、反例で確かめ、説明を更新する経験を積みました。

次の開発でも、判断を裏付ける証拠を残してください。

# 参考

- 本教材: `examples/15-showcase/`
- 第1～14回の仕様・演習・振り返り
- [The Scheme Programming Language](#)

外部の言語や処理系と比較するときは、機能差を明記してください。